

Thibaut Waechter

Année universitaire : 2024–2025

Session : Septembre 2025

DEVELOPPEMENT D'UN ESSAIM DE ROBOTS MOBILES

Spécialité Génie Electrique - Parcours S-IoT

Ce travail s'inscrit dans le cadre du projet eSWARM, initié en 2024, dont l'objectif est de développer un système de drones quadricoptères modulaires et reconfigurables, capables de s'assembler et de se désassembler en vol pour former différentes structures.

Afin de faciliter les expérimentations, une plateforme de robots terrestres holonomes a été mise en place en substitut aux drones aériens. Sur cette base, un algorithme distribué de contrôle d'essaim a été développé sous ROS2 en Python, permettant le pilotage collaboratif de plusieurs robots.

La commande a ensuite été optimisée par l'implémentation de paradigmes de contrôle événementiel, incluant un déclenchement événementiel classique (mise à jour conditionnelle des commandes) et un déclenchement prédictif (échanges de données conditionnels). Des outils logiciels d'acquisition et d'enregistrement de données expérimentales ont également été conçus, ouvrant la voie à une analyse quantitative des performances.

Organisme d'accueil : iCube

300 Boulevard Sébastien Brandt

67400 Illkirch

Sylvain Durand

sylvain.durand@insa-strasbourg.fr

03/03/25 – 05/09/25



1 Introduction et contexte du stage

Ce travail s'inscrit dans la continuité du projet eSWARM, dont l'objectif est de développer un système de drones quadricoptères modulaire et reconfigurable. Les appareils doivent être capables de s'assembler en vol afin d'accomplir des missions qu'un drone isolé ne pourrait réaliser, comme le transport de charges lourdes [1]. Les systèmes robotiques modulaires et reconfigurables ont été définis dans [2, 3]. Ils reposent sur une architecture composée de modules pouvant s'assembler dans différentes configurations et se réorganiser dynamiquement en fonction de la tâche à accomplir.

Parmi les projets de référence, le système ModQuad [4] illustre la faisabilité de telles approches en permettant à des quadricoptères de former des structures plus complexes par assemblage. La difficulté de ce type de système est la conception des lois de commande permettant de piloter une structure dont la configuration peut changer au cours du temps.

Dans ce contexte, et afin de réduire les contraintes expérimentales liées aux drones (autonomie limitée, risque de crash, fragilité du matériel), une alternative reposant sur des robots terrestres holonomes (pouvant se déplacer sans contrainte dans le plan) a été mise en place pour tester les algorithmes. L'objectif est ici de développer, implémenter et comparer différents algorithmes afin d'optimiser la commande d'un essaim de robots.

2 Méthodologie

2.1 Algorithme de contrôle d'essaim distribué

L'algorithme retenu est celui de [5], qui est un algorithme de consensus distribué. Il s'inspire grandement de [6], qui est utilisé pour piloter un essaim de drones, ce qui permet de rester dans la continuité du projet eSWARM. De plus, l'algorithme est relativement simple à mettre en place, puisqu'il est basé sur l'utilisation de PIDs. En contrôle distribué, le consensus désigne un mécanisme où chaque agent ajuste son état à partir de ceux de ses voisins de telle sorte que tous les états convergent vers une valeur commune, sans coordinateur central. Dans ce cadre, l'algorithme calcule deux contributions de commande pour chaque agent : u_i^α (formation), qui régule les distances inter-robots vers leurs valeurs désirées, et u_i^γ (attraction), qui déplace le robot vers un point cible. La commande appliquée est la somme des deux : $u_i = u_i^\alpha + u_i^\gamma$.

2.2 Optimisation de la commande par contrôle événementiel

Afin de réduire la charge de calcul et le trafic réseau sans dégrader la stabilité du consensus, deux variantes de contrôle événementiel ont été considérées. La première cherche à limiter le nombre de calcul de commandes locales (commande événementielle classique). La seconde vise à limiter le nombre d'information partagé entre les différents robots de l'essaim.

2.2.1 Commande événementielle classique

Cette stratégie consiste à ne recalculer les deux composantes de commande u_i^α (formation) et u_i^γ (attraction) uniquement lorsque c'est réellement utile. Concrètement, chaque robot surveille deux indicateurs simples :

- δ_d (formation) : mise à jour si, pour au moins un voisin j , l'écart de distance dépasse le seuil

$$|d_{ij} - d_{ij}^*| > \delta_d$$

- δ_c (cible) : mise à jour à l'approche de la cible, ce qui permet au robot de ne pas dépasser la cible

$$\|\mathbf{p}_i - \mathbf{p}_i^*\| < \delta_c$$

- *Condition non seuil* : changement de la cible globale $\mathbf{g}$

Tant que ces écarts restent sous des seuils fixés, la dernière commande est conservée; dès qu'un seuil est dépassé, une mise à jour est déclenchée. Cette logique évite des calculs inutiles tout en maintenant la formation et la progression vers la cible.

2.2.2 Commande événementielle prédictive

La commande événementielle prédictive a pour objectif de limiter les échanges d'informations entre les robots. La logique est la suivante : chaque robot prédit la position de ses voisins à partir de leur dernière position publiée et de leur vitesse et utilise cette prédiction pour calculer sa propre commande. Une nouvelle position n'est publiée que si l'erreur de prédiction dépasse un certain seuil.

Chaque robot utilise la même méthode de prédiction pour ses voisins : il estime leur position actuelle en ajoutant à leur dernière position publiée le produit de leur vitesse et du temps écoulé. Chaque robot connaît sa propre position réelle (mesurée) et sa position prédite (calculée avec la même méthode que pour ses voisins). Il peut donc comparer ces deux valeurs et déterminer précisément quand la prédiction devient trop imprécise, ce qui déclenche la publication de sa vraie position.

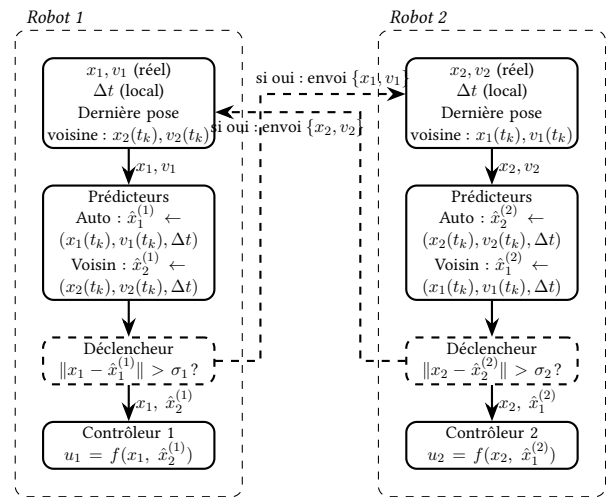


FIGURE 1 – Schéma de la commande prédictive

2.2.3 Outils utilisés

Les principaux outils mobilisés durant le stage sont les suivants :

- Robots TurtleBot à roues Mecanum (holonomes), pilotés par des servomoteurs Dynamixel via une carte OpenCR et une Raspberry Pi 4, utilisés comme substituts des drones pour les expérimentations initiales;
- ROS2 comme middleware de communication et d'orchestration, avec lequel les algorithmes de contrôle en Python ont été programmés;
- Système de motion capture OptiTrack/Motive, permettant d'obtenir en temps réel la position des robots via VRPN.

Les programmes réalisés sont disponibles sur le dépôt GitHub du projet : github.com/watson67/mecanum. Aussi, un ensemble de noeuds ROS2 ont été développés pour enregistrer les données expérimentales sous forme de fichier CSV.

3 Résultats

Les différentes méthodes ont été testé en faisant réaliser à un essaim de 4 robots le même parcours, à savoir un carré en partant d'une position initiale fixée. Un échantillon de résultat pour la méthode classique est donnée ci-dessous à titre d'exemple. Les robots doivent maintenir 50cm d'inter-distance avec leurs voisins, en formant une formation carré. Deux robots sont considérés comme voisins s'ils sont adjacents et forment un côté du carré. Une perturbation est également créée en déplaçant un robot à la main durant l'expérience.

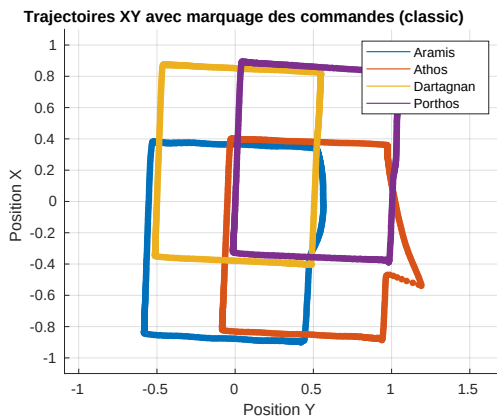


FIGURE 2 – Trajectoire des robots avec une perturbation

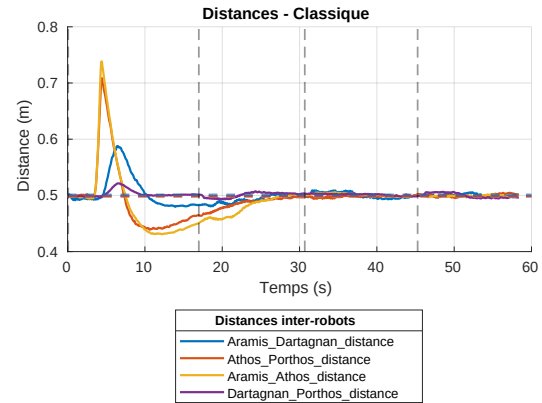


FIGURE 3 – Evolution des inter-distances entre les robots avec perturbation

Voici un tableau récapitulatif des résultats obtenus durant ce stage :

TABLE 1 – Tableau récapitulatif des performances par méthode

Critère	Classique	Événementielle	Prédictive
Temps de parcours [s]	48	39	56
Temps de convergence [s]	13	26	15
Nombre total de commandes calculées	1932	442	2187
Calculs de commande par seconde	≈ 40.3	≈ 11.3	≈ 39.1
Nombre total d'échanges de données	966	780	110
Échanges de données par seconde	≈ 20.1	20	≈ 1.96
Utilisation du processeur [%]	≈ 20	≈ 20	≈ 15
Utilisation de la mémoire vive [Mo]	≈ 470	≈ 470	≈ 470

En conclusion, les résultats obtenus sont satisfaisants et chaque méthode a répondu à ses objectifs principaux.

La méthode prédictive a réduit considérablement le nombre d'échanges de données tout en maintenant des performances de suivi de trajectoire similaires à la méthode classique. De plus, elle a permis de diminuer la charge CPU. L'intérêt de cette méthode est donc prouvé, particulièrement dans le contexte d'un essaim de grande taille. En effet, la réduction des communications est cruciale pour le passage à l'échelle : elle limite la surcharge du réseau, réduisant ainsi le risque de problèmes lié au réseau.

La méthode événementielle a quant à elle bien réduit le nombre de calculs de commande. Cepen-

dant, l'intérêt de cette approche est plus discutable, car la charge CPU n'a pas diminué par rapport à la méthode classique. Pour évaluer un potentiel gain, il aurait été pertinent de mesurer la consommation énergétique des moteurs. L'hypothèse serait que le maintien d'une vitesse constante plus fréquente réduirait la consommation, bien que les accélérations brusques lors des mises à jour de commande puissent avoir l'effet inverse. Malheureusement, cette mesure n'a pas été possible durant le stage, faute de capteurs de courant.

La prochaine étape du projet consistera à transposer ces algorithmes sur des drones quadricoptères, en s'appuyant sur les enseignements tirés de cette phase expérimentale avec les robots terrestres holonomes.

Bibliographie

- [1] Augustin BONNEL. *Modeling and control of a modular system of UAVs*. Rapport de PFE. Strasbourg : INSA, 2024, p. 60. (Visité le 03/03/2025).
- [2] Jungwon SEO, Jamie PAIK et Mark YIM. « Modular Reconfigurable Robotics ». In : *Annual Review of Control, Robotics, and Autonomous Systems 2* (Volume 2, 2019 3 mai 2019). Publisher : Annual Reviews, p. 63-88. ISSN : 2573-5144. DOI : 10.1146/annurev-control-053018-023834. URL : <https://www.annualreviews.org/content/journals/10.1146/annurev-control-053018-023834> (visité le 10/03/2025).
- [3] Sam BROOKS et Rajkumar ROY. « An overview of self-engineering systems ». In : *Journal of Engineering Design* 32.8 (3 août 2021). Publisher : Taylor & Francis _eprint, p. 397-447. ISSN : 0954-4828. DOI : 10.1080/09544828.2021.1914323. URL : <https://doi.org/10.1080/09544828.2021.1914323> (visité le 10/03/2025).
- [4] David SALDAÑA et al. « ModQuad : The Flying Modular Structure that Self-Assembles in Mid-air ». In : *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 2018 IEEE International Conference on Robotics and Automation (ICRA). ISSN : 2577-087X. Mai 2018, p. 691-698. DOI : 10.1109/ICRA.2018.8461014. URL : <https://ieeexplore.ieee.org/document/8461014/?arnumber=8461014> (visité le 04/03/2025).
- [5] Daravuth KOUNG et al. « Consensus-based formation control and obstacle avoidance for non-holonomic multi-robot system ». In : *2020 16th International Conference on Control, Automation, Robotics and Vision (ICARCV)*. 2020 16th International Conference on Control, Automation, Robotics and Vision (ICARCV). Déc. 2020, p. 92-97. DOI : 10.1109/ICARCV50220.2020.9305426. URL : <https://ieeexplore.ieee.org/abstract/document/9305426> (visité le 09/04/2025).
- [6] Osamah SAIF, Isabelle FANTONI et Arturo ZAVALA-RÍO. « Distributed integral control of multiple UAVs : precise flocking and navigation ». In : *IET Control Theory & Applications* 13.13 (3 sept. 2019). Publisher : The Institution of Engineering and Technology, p. 2008-2017. DOI : 10.1049/iet-cta.2018.5684. URL : <https://digital-library.theiet.org/doi/10.1049/iet-cta.2018.5684> (visité le 06/06/2025).